Great, Vivek — continuing the manual. Here is **CHAPTER 23**, fully expanded, deeply detailed, and formatted for Word/PDF-style clarity.

==========

==========

CHAPTER 23 — INTERRUPTS, EXCEPTIONS & MMU INTERACTION

==========

==========

A maximum-detail, RTOS/QNX-style breakdown of how ARMv8 interrupts and exceptions interact with the MMU, how faults propagate across EL0/EL1/EL2/EL3, how GIC virtualization works, and how memory translation affects interrupt delivery.

This chapter explains **the full relationship between interrupts, exceptions, and the MMU** — a topic that is often misunderstood but absolutely critical for OS kernels, hypervisors, and RTOS systems.

****

****

23.1 — Why Interrupt/MMU Interaction Matters

****

****

Interrupts and exceptions depend heavily on MMU behavior:

- faults use MMU translations
- exception vectors must be mapped correctly
- interrupt controllers (GIC) use MMIO
- hypervisors virtualize interrupts
- page faults must be delivered to the correct EL
- TLB invalidation affects interrupt latency
- cacheability affects interrupt register access

Incorrect MMU configuration → broken interrupt system.

****

****

23.2 — Exception Levels and MMU Context

****

****

ARMv8 has four exception levels:

EL	Role	MMU Context
EL0	User	TTBR0_EL1
EL1	Kernel/OS	TTBR0/TTBR1_EL1
EL2	Hypervisor	TTBR0_EL2
EL3	Secure Monitor	Secure MMU context

Key Point

Exceptions always execute using the MMU context of the target EL, not the EL where the fault occurred.

Example: A user-space page fault → exception taken to EL1 → EL1's TTBR1_EL1 mappings must be valid.

****

****

23.3 — Exception Vector Mapping

****

****

Exception vectors must be:

- mapped in TTBR1_EL1 (kernel)
- executable
- readable
- Normal WB
- Inner Shareable
- aligned

Common Mistake

Mapping exception vectors as Device memory → CPU cannot fetch instructions → crash.

****

****

23.4 — Types of Exceptions Related to MMU

****

****

Instruction Abort

- Stage-1 or Stage-2 translation failure during fetch

Data Abort

- Stage-1 or Stage-2 translation failure during load/store

Synchronous Exceptions

- page faults
- alignment faults
- permission faults
- execute faults

Asynchronous Exceptions

- interrupts
- SError (system error)

****

****

23.5 — MMU Fault Delivery Path

****

****

Stage-1 Fault (EL0/EL1)

Delivered to **EL1**.

Stage-2 Fault (EL1 guest)

Delivered to **EL2**.

Secure Fault

Delivered to **EL3**.

Flow Example

```
EL0 → VA access
    ↓
Stage-1 fault → EL1 exception

EL1 guest → IPA access
    ↓
Stage-2 fault → EL2 exception
```

****

****

23.6 — ESR_ELx and Fault Decoding

****

****

Interrupt and exception handlers must decode:

- **EC** (Exception Class)
- **DFSC/IFSC** (Fault Status Code)
- **FAR_ELx** (Fault Address)
- **HPFAR_EL2** (IPA for Stage-2 faults)

These registers determine:

- fault type
- fault stage
- fault level
- faulting address
- required recovery action

****

****

23.7 — Interrupt Controller (GIC) MMU Requirements

****

****

GIC components (GICD, GICR, GICC, GITS) must be mapped:

- Device-nGnRnE
- Non-Shareable
- 4 KB pages
- AP=RW
- PXN/UXN

Why?

Interrupt registers require strict ordering and non-cacheable access.

Common Mistake

Mapping GIC as Normal memory → interrupts behave unpredictably.

****

****

23.8 — Interrupt Latency and MMU

****

****

Interrupt latency depends on:

- TLB hit rate
- page-table walk latency
- cacheability of vector table
- cacheability of GIC registers
- DSB/ISB usage
- SMP coherency

Best Practices

- map vector table as Normal WB
- map GIC as Device-nGnRnE
- avoid TLB misses in interrupt path
- avoid page faults in interrupt path

****

****

23.9 — Interrupts During Page Faults

****

****

Interrupts may occur:

- during page-table walks
- during TLB invalidation
- during BBM sequences
- during descriptor updates

Rules

- interrupts do NOT bypass MMU
- interrupt handlers must use valid TTBR1_EL1 mappings
- hypervisor must ensure Stage-2 mappings for GIC are stable

****

****

23.10 — Interrupts and TLB Invalidation

****

****

TLBI affects interrupt latency:

- TLBI may stall pipeline
- TLBI may delay interrupt entry
- TLBI must be followed by DSB SY
- TLBI must be followed by ISB

Hypervisor Impact

TLBI ALLE2 may delay guest interrupts.

****

****

23.11 — Interrupts and Cache Maintenance

****

****

Cache maintenance affects interrupt behavior:

- cleaning MMIO is illegal
- invalidating MMIO is illegal
- cleaning vector table is safe
- invalidating vector table is safe

Best Practice

Never run DC operations on GIC registers.

****

****

23.12 — Interrupts and Shareability

****

****

Shareability affects interrupt coherency:

Inner Shareable

- required for SMP
- ensures vector table coherence
- ensures TLB broadcast

Non-Shareable

- required for MMIO
- ensures device ordering

****

****

23.13 — Interrupts and Stage-2 Virtualization

****

****

Hypervisor virtualizes:

- interrupt routing
- interrupt masking
- interrupt injection
- virtual CPU interfaces

GIC Virtualization Layers

- GICv2 → VGIC
- GICv3 → GICR virtualization
- GICv4 → direct injection

Stage-2 Requirements

- GIC MMIO must be mapped in Stage-2
- S2AP must allow guest access
- S2PXN/S2UXN must block execution

****

****

23.14 — Interrupts and Nested Virtualization

****

****

Nested virtualization requires:

- nested interrupt routing
- nested VGIC
- nested Stage-2 faults
- nested MMIO emulation

Complexity

Interrupt virtualization becomes significantly more expensive.

****

****

23.15 — Exceptions and Memory Ordering

****

****

Exceptions require strict ordering:

Before reading fault registers

```
DSB SY
```

Before returning from exception

```
ISB
```

Why?

Ensures pipeline sees updated state.

****

****

23.16 — Exceptions and Device Memory

****

****

Device memory exceptions include:

- alignment faults
- access faults
- permission faults

Rules

- device memory must be Non-Shareable
- device memory must be Device-nGnRnE
- device memory must use 4 KB pages

****

****

23.17 — Exceptions and QNX/RTOS

****

****

QNX requires:

- deterministic exception handling
- static vector table
- static MMIO mappings
- strict partition isolation
- predictable interrupt latency

Best Practices

- avoid page faults in safety partitions
- avoid dynamic remapping
- avoid dynamic MMIO mapping

****

****

23.18 — Common Interrupt/MMU Mistakes

****

****

Mistake 1 — Mapping GIC as Normal WB

Interrupts malfunction.

Mistake 2 — Page faults in interrupt path

Unbounded latency.

Mistake 3 — Missing DSB after MMIO writes

Device state corrupted.

Mistake 4 — Missing ISB after MMU changes

Pipeline uses stale translations.

Mistake 5 — Incorrect Stage-2 GIC mapping

Guest interrupts fail.

****

****

23.19 — Full Interrupt/MMU Example

****

****

GIC Mapping

```
AttrIndx = 2 (Device-nGnRnE)
SH = 00 (Non-Shareable)
AP = RW
PXN = 1
UXN = 1
```

Interrupt Entry

```
DSB SY
Read GIC registers
```

Interrupt Exit

```
ISB
ERET
```

****

****

23.20 — Chapter Summary

****

****

This chapter covered:

- exception levels and MMU context
- exception vector mapping
- MMU fault delivery
- ESR/FAR/HPFAR decoding
- GIC mapping rules
- interrupt latency and MMU
- interrupts during page faults
- TLB invalidation impact
- cache maintenance impact
- shareability impact
- Stage-2 interrupt virtualization
- nested virtualization
- device memory exceptions
- QNX/RTOS behavior
- common mistakes
- full examples

This prepares the foundation for **Chapter 25**, which covers **Boot Process & MMU Bring-Up (EL3 → EL2 → EL1)** in maximum detail.

⭐ Next Message: CHAPTER 25 — BOOT PROCESS & MMU BRING-UP (EL3 → EL2 → EL1)